

VorodipYampsid — ваша необходимость наша задача

IT решение для управления digital
инфраструктурой жилого комплекса



Ключевая задача

Объединить все дисплеи в единую сеть
для показа новостей, рекламы и
оповещений об чрезвычайных
ситуациях



**Проблемы которые
мы решаем**

**Непредсказуемое
поведение
дисплеев**



**Проблемы которые
мы решаем**

**Требования к
квалификации
маркетологов**



**Проблемы которые
мы решаем**

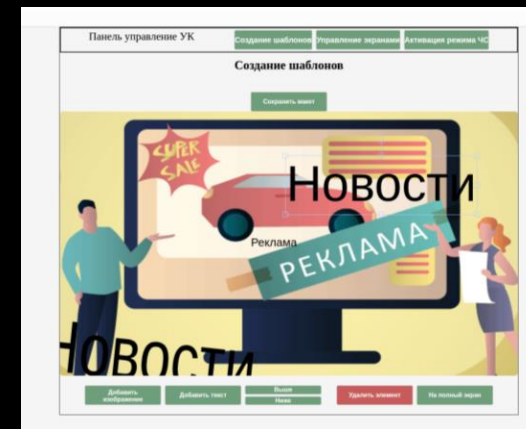
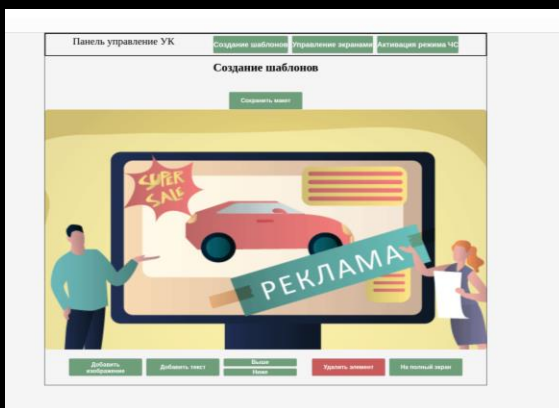
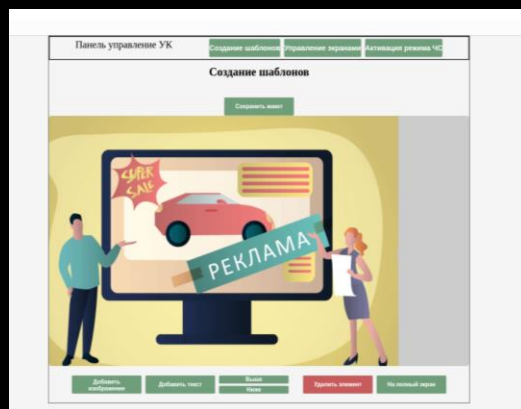
**Высокие
сопутствующие
расходы**



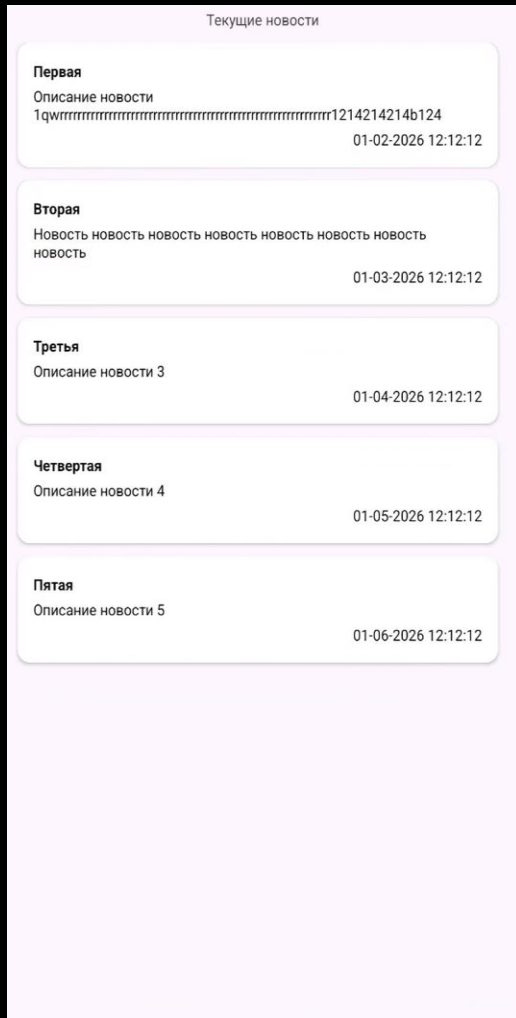
Как?

Admin панель

Прототип интерфейса для создания банеров



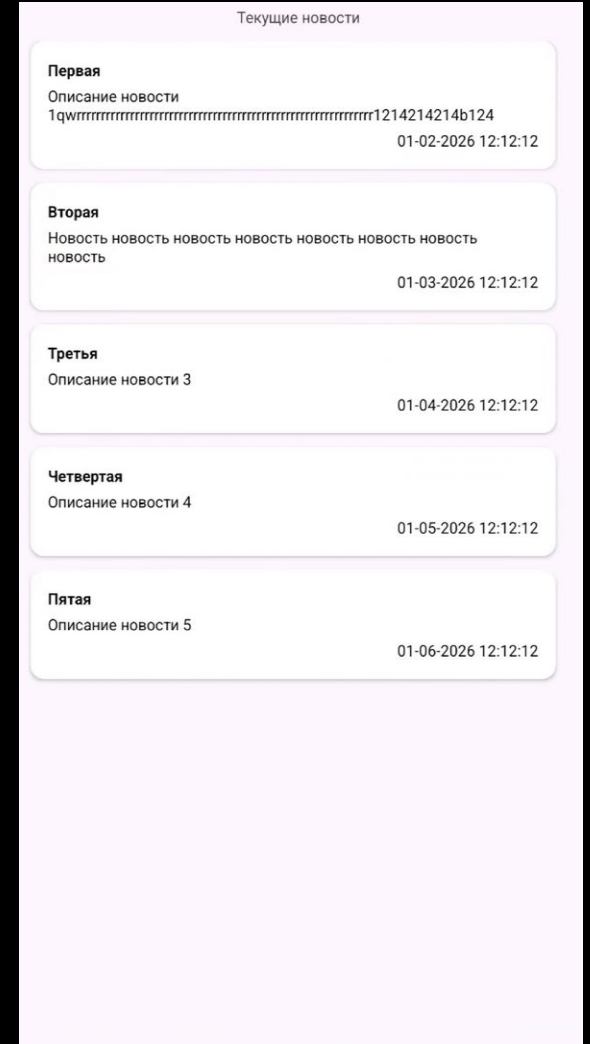
Приложение для дисплеев



20 сек.



20 сек.



Масштабируемый сервер

```
ntext.cs  appsettings.json  PosterController.cs  EmployeeController.cs  Program.cs  AuthService.cs  Settings  JwtService.cs  IAuthService.cs
on > backend > Middleware > Services > StorageService.cs > StorageService

public class StorageService : IStorageService
{
    public async Task<GetPostersResponse> SendListCanvas(PlaginationMeta plagination)
    {
    }
    2 references
    public async Task<GetPostersAndroidResponse> SendImagesToAndroid(int deviceId)
    {
        try
        {
            using var db = new LiteDatabase(_context.PathToDatabase);
            var devicePoster = db.GetCollection<DevicePoster>("devicePoster");
            var necessaryIdentificators = devicePoster.Query()
                .Where(dp => dp.DeviceId == deviceId)
                .Select(dp => dp.PosterId)
                .ToList();

            var posters = db.GetCollection<Poster>("poster");

            var requiredPosters = posters.Query()
                .Where(p => necessaryIdentificators
                    .Contains(p.Id))
                .ToList();

            List<byte[]> images = new List<byte[]>();
            foreach(var poster in requiredPosters)
            {
                images.Add(await GetByteImageAsync(posters.PngLink));
            }

            GetPostersAndroidResponse response = new GetPostersAndroidResponse()
            {
                Images = images,
            };
            return response;
        }
        catch
        {
            throw new ApiException(500, "Внутренняя ошибка сервера");
        }
    }
}

2 references
private string GenerateUniqueFileName(PosterTypes types)
```

```

[HttpGet]
[Route("android/{id}")]
0 references
public async Task<IActionResult> SendPosterToAndroid()
{
    try
    {
        string? id = User?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        if(!int.TryParse(id, out int deviceId))
        {
            return StatusCode(401, "id user не указан");
        }
        var response = await _storageService.SendImagesToAndroid(deviceId);
        return Ok(response);
    }
    catch(Exception ex)
    {
        if(ex is ApiException apiEx)
        {
            int statusCode = apiEx.StatusCode;
            return StatusCode(statusCode, apiEx.Info);
        }
        return StatusCode(500, "Неожиданная ошибка сервера");
    }
}
}
```

Стек